Python Activity-Based Learning

Case Study 49: Tuple-Based Product Catalogue Audit

1. Problem Statement

Store product identifiers, prices, and stock quantities in tuples. Detect invalid records, calculate inventory values, and identify products that need replenishment.

2. Objectives

- Represent fixed product records using tuples.
- Unpack tuple values in a loop.
- Calculate stock value and detect low stock.

3. Assumptions

- Each product tuple is (product_id, price, quantity).
- Price and quantity must be non-negative.
- Products with fewer than 5 units in stock need replenishment.
- The data is entered directly in the program for this exercise.

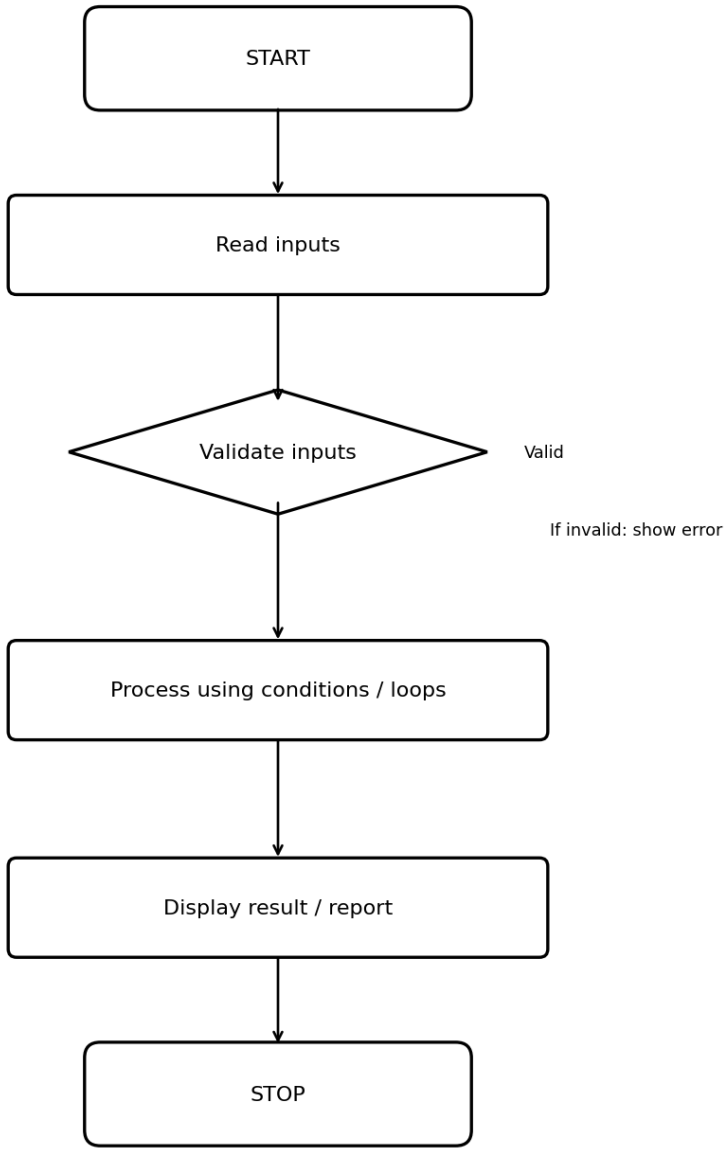
4. Algorithm

1. Create a list of product tuples.
2. Unpack each tuple into identifier, price, and quantity.
3. Check for invalid values and duplicate identifiers.
4. For valid records, calculate inventory value and add it to the total.
5. Flag products with stock below five units.
6. Display record status and total inventory value.

5. Flowchart

The diagram below shows the overall program flow. The validation decision represents the input checks described in the algorithm.

Flowchart: Tuple-Based Product Catalogue Audit



6. Pseudocode

```
START
products = list of tuples (id, price, quantity)
CREATE empty list seen_ids
total_value = 0
FOR each tuple in products
    UNPACK id, price, quantity
    IF id is already in seen_ids THEN DISPLAY duplicate id
    ELSE add id to seen_ids
    IF price < 0 OR quantity < 0 THEN DISPLAY invalid record
    ELSE
        value = price * quantity
        total_value = total_value + value
        DISPLAY value
        IF quantity < 5 THEN DISPLAY replenishment required
```

```
    END IF
END FOR
DISPLAY total inventory value
STOP
```

7. Python Source Code

The solution uses only basic Python features from the assigned syllabus.

```
products = [
    ("P101", 50.0, 10),
    ("P102", 120.0, 3),
    ("P103", 75.0, 0),
    ("P102", 90.0, 4)
]

seen_ids = []
total_value = 0

print("PRODUCT CATALOGUE AUDIT")
for product in products:
    product_id, price, quantity = product  # tuple unpacking

    if product_id in seen_ids:
        print(f"{product_id}: duplicate product ID")
    else:
        seen_ids.append(product_id)

    if price < 0 or quantity < 0:
        print(f"{product_id}: invalid price or quantity")
    else:
        value = price * quantity
        total_value += value
        print(f"{product_id}: price=₹{price:.2f}, stock={quantity}, "
              f"inventory value=₹{value:.2f}")
        if quantity < 5:
            print(f"{product_id}: replenishment required")

print(f"Total inventory value (valid numeric records): ₹{total_value:.2f}")
```

8. Test Cases

Test input / condition	Expected result	Purpose
P101, ₹50, qty 10	Value ₹500	No replenishment
P102, ₹120, qty 3	Value ₹360	Replenishment required
Repeated ID P102	Duplicate identifier	Duplicate message displayed

9. Sample Output Screenshot

Sample console output for one test run:

```
C:\Users\Badri\PycharmProjects\PythonProject-1\.venv\Scripts\python.exe "C:\Users\Badri\PycharmProjects\PythonProject-1\Case Studies\case
 study - 3.py"
PRODUCT CATALOGUE AUDIT
P101: price=₹50.00, stock=10, inventory value=₹500.00
P102: price=₹120.00, stock=3, inventory value=₹360.00
P102: replenishment required
P103: price=₹75.00, stock=0, inventory value=₹0.00
P103: replenishment required
P102: duplicate product ID
P102: price=₹90.00, stock=4, inventory value=₹360.00
P102: replenishment required
Total inventory value (valid numeric records): ₹1220.00

Process finished with exit code 0
```

10. Conclusion

This case study demonstrates tuples, tuple unpacking, list membership checks, loops, and stock-value calculations.

Note: Assumed rates, thresholds, and sample values are for this educational exercise and can be changed if the instructor supplies different rules.